

## demo man

Note : si vous voulez que tous les exemples de cette démo fonctionnent sur votre système, vous pouvez installer les paquets `poppler-utils figlet cowsay`.

## Introduction

Le manuel est un ensemble de pages de documentation directement installés sur le système, ainsi qu'un logiciel permettant d'y accéder.

Se documenter à travers une commande peut sembler archaïque lorsqu'on a disposition le web ou des chatbots LLM. Néanmoins, ces deux méthodes demandent un effort supplémentaire pour ne pas tomber dans les cultes du cargo, en copiant/collant une « solution » dont les effets peuvent être catastrophiques (un même symptôme peut avoir plusieurs causes).

Voici quelques avantages qu'il y a à maîtriser l'utilisation du manuel :

- Disponibilité : le manuel est installé sur le système local, donc disponible même si internet est coupé ou filtré.
- Vitesse : le manuel est accessible depuis le même shell que celui que vous utilisez pour exécuter des commandes. Si vous voulez juste vous remémorer la syntaxe d'une option, vous ne perdez pas de temps à changer de fenêtre.
- Contrôle : utiliser le manuel demande de comprendre ce que l'on est en train de faire puisqu'on fabrique soi-même une commande à partir d'un descriptif des options. Vous gardez ainsi le contrôle de ce que vous exécutez (parfois en tant qu'utilisateur `root`).
- Compatibilité : une page du manuel est fournie avec le paquet du logiciel qu'il documente. Elle est donc à jour et cohérente avec la version du logiciel installé. Si une option se comporte différemment selon la version d'un logiciel, vous aurez le jeu d'options correspondant au logiciel que vous exécutez. Il n'est pas rare de devoir administrer ou réparer des systèmes vieux de plusieurs années ou décennies.

## Utilisation basique

Pour obtenir de l'aide sur un programme exécutable (ou autre comme on verra plus bas), il suffit d'exécuter

```
$ man <programme>
```

Par exemple, imaginez que vous voulez créer une suite de sous-répertoires `/tmp/a/b/c/d/e`. La commande `mkdir` utilisée telle quelle ne le permet pas de le faire directement :

```
$ mkdir /tmp/a/b/c/d/e
mkdir: impossible de créer le répertoire « /tmp/a/b/c/d/e »: Aucun fichier ou dossier de ce type
```

En effet, la commande `mkdir /tmp/a/b/c/d/e` est censée créer un répertoire nommé `e` dans le répertoire `/tmp/a/b/c/d` qui est donc censé déjà exister.

Ainsi, si seul `/tmp` existe, il faudrait créer les répertoires les uns après les autres comme ceci

```
$ mkdir /tmp/a
$ mkdir /tmp/a/b
$ mkdir /tmp/a/b/c
$ mkdir /tmp/a/b/c/d
```

```
$ mkdir /tmp/a/b/c/d/e
```

On se dit que ce genre de cas est fréquent et que le problème a probablement déjà été résolu. Regardons le manuel de la commande `mkdir` :

```
$ man mkdir
```

Cette page de manuel est courte et traduite en français, on identifie rapidement l'option `-p` ou `--parents` qui permet de créer les répertoires parents dans le bon ordre. Ainsi, les 5 commandes précédentes peuvent être remplacées par la commande

```
$ mkdir -p /tmp/a/b/c/d/e
```

Au passage, on voit que l'option a deux syntaxes possibles, d'une part `--parents` avec deux tirets est explicite et très lisible (utile pour des scripts partagés), d'autre part `-p` avec un seul tiret est une version raccourcie de la première (utile en ligne de commande pour gagner du temps).

## Naviguer dans une page du manuel

La commande `man` délègue l'affichage des pages du manuel à un *pager* (fr: visionneur en mode terminal). Il s'agit généralement du programme `less` qui permet d'afficher et de naviguer facilement dans un fichiers texte.

Voici quelques raccourcis clavier à connaître, ils sont très proches de ceux de `vim` :

- `g` (ou `gg` si on veut utiliser les mêmes raccourcis que `vim`) : aller au début du document
- `G` : aller à la fin du document (les exemples, fichiers et références en lien se trouvent en bas du manuel)
- `<espace>` : page suivante
- `q` : quitter
- `/<motif>` : chercher le `<motif>` dans la suite du texte
  - `n` : aller à la prochaine occurrence du `<motif>`
  - `N` : aller à la précédente occurrence du `<motif>`
- `:<num>` va à la `<num>`-ième ligne
- `h` pour obtenir plus d'aide, voir aussi `man less` :)

Par exemple, supposons qu'on cherche à identifier les plus gros exécutable du système (qui se trouvent dans `/usr/bin`), on peut essayer de les lister en les rangeant par taille.

```
$ man ls
```

On peut naviguer dans la page du manuel, en cherchant le motif « taille » en tapant `/taille`. L'option `--block-size` ne semble pas intéressante, donc on la passe en tapant `n` deux fois, on découvre

- que l'option `-s` permet de lister la taille à côté des noms de fichiers
- que l'option `-S` permet de trier les fichiers par ordre de taille décroissante

```
$ ls -s -S /usr/bin/
```

Le listing est bien long et les plus gros fichiers sont sortis de l'écran. On pourrait sélectionner le début de la sortie en combinant avec la commande `head` :

```
$ ls -s -S /usr/bin/ | head -n 30
```

mais comme on est dans une démo sur le `man`, on peut aussi chercher à inverser l'ordre d'affichage. Avec `/invers` on trouve l'option `-r` ou `--reverse`.

Ainsi, la commande

```
$ ls -s -S -r /usr/bin
```

nous permet de voir quels sont les plus gros exécutable.

Si le terminal est large, on voit que la commande `ls` met plusieurs noms de fichiers sur la même ligne. Pour régler ce problème on peut chercher `/ligne` et découvrir rapidement que l'option `-1` permet d'afficher un fichier par ligne.

Aussi, une sortie du type 193524 `pandoc` demande de la concentration pour estimer la taille, car celle-ci est exprimée en nombre de blocs (et on a ignoré l'option `--block-size`). On se souvient avoir vu passer l'option `-h` ou `--human-readable` lorsqu'on cherchait le motif `taille`.

Bilan des courses, la commande suivante répond assez bien à notre problème, et il n'a pas été nécessaire de lire toute la page du manuel de `ls` pour la concevoir :

```
$ ls -s -S -r -h -1 /usr/bin
```

Sur le système de l'enseignant-e, l'exécutable le plus gros est `pandoc` et sa taille est de 189MiB.

Remarque : les options courtes (à un seul tiret) peuvent souvent se concaténer, ainsi, la commande précédente est équivalente à

```
$ ls -sSrh1 /usr/bin
```

On peut recycler cette commande pour les exécutables réservés à l'utilisateur `root`

```
$ ls -sSrh1 /usr/sbin
```

## Chercher parmi les pages du manuel

La commande

```
man -k <motif>
```

liste les pages du manuel dont la description courte contient `<motif>`. L'option `-K` (majuscule) permet de chercher dans le contenu complet des pages de manuel.

Par exemple, supposons qu'on cherche à transformer un fichier au format pdf en fichier texte.

On peut chercher les pages du manuel qui parlent de pdf :

```
$ man -k pdf
pdf2djvu (1)      - Créer des fichiers DjVu à partir des fichiers PDF
a2ping (1)       - - convert between PS, EPS and PDF and other page descr...
a5toa4 (1)       - arrange A5 pages in a PDF onto A4 landscape paper
afmtodit (1)     - create font files for use with groff -Tps and -Tpdf
any2djvu (1)     - Convert .ps/.ps.gz/.pdf to .djvu
axohelp (1)      - helper program for use of LaTeX package axodraw2 with ...
bg5+pdfplatex (1) - Use pdf directly on a Big5+ encoded .tex file
...
...
```

Mais (en tous cas sur le système de l'enseignant-e), on trouve plus de 100 commandes. On peut alors filtrer :

```
$ man -k pdf | grep text
comparepdf (1)   - compare two PDF files textually or visually
diffpdf (1)     - compare two PDF files textually or visually
ocrmypdf (1)    - add an OCR text layer to PDF files
pdfcslatex (1)  - structured text formatting and typesetting
pdftotext (1)   - Portable Document Format (PDF) to text converter (version 3.03)
```

On voit immédiatement que la commande `pdftotext` est une candidate crédible.

On peut alors se référer à la page de manuel correspondante pour :

```
$ man pdftotext
```

On peut alors transformer `fichier.pdf` en `fichier.txt` et vérifier que ce dernier est bien un fichier au format texte :

```
$ pdftotext fichier.pdf fichier.txt
$ less fichier.txt
```

## Sections

Le manuel ne documente pas que des exécutables.

Les pages de manuel sont organisées en sections selon qu'elles documentent un exécutable, un appel système, un fichier spécial, un fichier de configuration, une bibliothèque d'un langage de programmation, des conventions, etc.

La page de manuel de la commande `man` donne une description des sections. Ainsi si vous exécutez

```
$ man man
```

vous obtenez la liste :

1. Programmes exécutables ou commandes de l'interpréteur de commandes (shell)
2. Appels système (fonctions fournies par le noyau)
3. Appels de bibliothèque (fonctions fournies par les bibliothèques des programmes)
4. Fichiers spéciaux (situés généralement dans `/dev`)
5. Formats des fichiers et conventions. Par exemple `/etc/passwd`
6. Jeux
7. Miscellaneous (including macro packages and conventions), e.g. `man(7)`, `groff(7)`, `man-pages(7)`
8. Commandes de gestion du système (généralement réservées au superutilisateur)
9. Sous-programmes du noyau [hors standard]

Par exemple, il existe une commande `passwd` permettant de modifier le mot de passe d'un user, mais aussi un fichier de configuration `/etc/passwd`.

On peut les distinguer en précisant la section concernée :

```
$ man 1 passwd
```

```
$ man 5 passwd
```

Remarque : on peut évidemment combiner et chercher un motif dans une section précise :

```
$ man -s <section> -k <motif>
```

## Observation des sections

On peut faire un tour rapide des sections en regardant quelques exemples dans chacune d'elles.

1. On a déjà vu le manuel des exécutables `mkdir` et `ls` plus haut.
2. Vous verrez les appels systèmes en cours de système au prochain semestre. Il s'agit d'une instruction qu'un programme peut utiliser pour demander au noyau de faire quelque chose pour lui. Par exemple seul le noyau peut créer des nouveaux processus, cela se fait avec l'appel système `fork` sur lequel vous pouvez obtenir de la documentation (avec des exemples) grâce à :

```
$ man 2 fork
```

Notez la différence entre appel système et exécutable :

```
$ man 2 chmod
$ man 1 chmod
```

Ici, la commande `chmod` (que l'on peut exécuter depuis un shell) va utiliser l'appel système `chmod` (plus précisément son variant `fchmodat`) lors de son exécution, car seul le noyau est capable de modifier les permissions d'un fichier.

3. Vous pouvez vous documenter sur la bibliothèque standard `stdio` du langage C :

```
$ man 3 stdio
```

Cette page de manuel liste en particulier toutes les fonctions fournies par cette bibliothèque, et chaque fonction possède sa propre page de manuel avec de nombreux exemples, par exemple la fonction `printf` que vous connaissez bien :

```
$ man 3 printf
```

On voit par exemple que la `stdio` permet aussi créer des fichiers temporaires sans avoir à se souvenir de leur trouver un nom unique :

```
$ man 3 tmpfile
```

4. Il s'agit de fichiers spéciaux, qu'on verra lors du prochain cours, par exemple un fichier toujours vide ou un fichier qui produit une infinité de zéros :

```
$ man 4 null
```

```
$ man 4 zero
```

5. De nombreux fichiers de configurations (qui se trouvent dans `/etc`) ont une structure particulière qui est décrite dans leur page du manuel, par exemple

```
$ man 5 fstab
```

documente le fichier `/etc/fstab` qui permet entre autres de décrire les systèmes de fichiers à monter dans l'arborescence au démarrage du système (cf cours à venir).

```
$ man 5 sources.list
```

documente le fichier `/etc/apt/sources.list` qui liste les miroirs pour le gestionnaire de paquets `apt`.

```
$ man 6 ssh_config
```

documente simultanément le fichier de config de SSH au niveau du système `/etc/ssh/ssh_config` ainsi que les fichiers de config de SSH au niveau d'un user `~/.ssh/config` (c'est la même syntaxe).

6. Même les jeux ont droit à leurs pages de manuel.

Notez que l'autocomplétion fonctionne pour la commande `man`. Par exemple, si on tape :

```
$ man 6 [TAB] [TAB]
```

on voit apparaître toutes les pages de manuel de la section 6, on voit par exemple

```
$ man 6 cowsay
```

qui fait parler une vache :

```
$ cowsay BOUM
```

ou encore

```
$ man 6 figlet
```

qui écrit avec de gros caractères en ASCII-art :

```
$ figlet BOUM
```

On peut essayer de les combiner pour voir :

```
$ figlet BOUM | cowsay
```

mais le rendu est pas super car les espaces consécutifs sont remplacés par un seul espace. Peut-être qu'on peut réparer ça ?

```
$ man 6 cowsay
```

l'option `-n` semble adaptée :

```
$ figlet BOUM | cowsay -n
```

C'est l'histoire d'une vache qui parle d'une vache qui parle d'une vache qui dit BOUM (réduire la fonte du terminal pour avoir de l'espace) :

```
$ figlet BOUM | cowsay -n | cowsay -n | cowsay -n
```

7. Pour se cultiver :

- Accéder rapidement à la table des caractères ASCII

```
$ man 7 ascii
```

- Les répertoires classiques d'une arborescence (un peu daté mais traduit en français)

```
$ man 7 hier
```

- Les répertoires classiques d'une arborescence (plus moderne et concis, mais en anglais)

```
$ man 7 file-hierarchy
```

- Interfaces permettant d'accéder à l'aléa du système

```
$ man 7 random
```

- La séquence de boot

```
$ man 7 boot
```

- Questions fréquentes à propos de git

```
$ man 7 gitfaq
```

- Les attributs d'inode

```
$ man 7 inode
```

- base 1000 ou 1024 ?

```
$ man 7 units
```

8. c'est le coin des sysadmin-es \o/

- gestionnaire des paquets

```
$ man 8 apt
```

- création d'users :

```
$ man 8 adduser
```

- partitionnement d'un block device :

```
$ man 8 parted
```

- formatage d'un système de fichiers

```
$ man 8 mkfs
```

- montage d'un système de fichiers

```
$ man 8 mount
```

- gestion du réseau

```
$ man 8 ip
```

## Remarque : primitives ou mots-clefs du shell

Comme on l'a vu dans la démo sur le shell, certaines commandes ne correspondent pas à des exécutables mais sont des primitives ou des mots clefs du shell.

Leur comportement dépend donc du shell courant et il ne peut pas y avoir de page de manuel qui leur correspondent au niveau du système :

```
$ man for
Aucune entrée de manuel pour for
```

```
$ man cd
Aucune entrée de manuel pour cd
```

La documentation de

```
$ help for
$ help cd
```

Et forcément

```
$ help help
```

## Remarque : découvrir des logiciels non-installés

Notez que comme chaque paquet installe les pages de manuel qui lui correspondent (exécutables, fichiers de config, etc), la commande **man -k** ne permet que de (re)découvrir les logiciels déjà installés sur le système ou de retrouver le nom d'une commande.

C'est utile lorsque vous débarquez sur un système existant et que vous avez besoin de savoir ce qui est disponible.

Le manuel ne permet pas de découvrir de nouveaux logiciels. Pour découvrir des logiciels installables par la distribution, vous pouvez utiliser la commande **apt search**.

## Dialogue de fin

- (Débutant.e) tu peux m'expliquer comment on configure le réseau ?
- (Expert.e) **man ip**
- (Débutant.e) mais je ne sais pas me servir du man !
- (Expert.e) **man man**